

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ,
МЕХАНИКИ И ОПТИКИ»**

Факультет безопасности информационных технологий

Дисциплина:
«Алгоритмы и структуры данных»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3
Топологическая сортировка с помощью алгоритма Демукрона

Выполнил:
Студент гр. N3248
Жук Анастасия Валерьевна

Подпись: _____

Проверил:
Ерофеев Сергей Анатольевич

Подпись: _____

Санкт-Петербург
2025 г.

Техническое задание

Разработать программу топологической сортировки с помощью алгоритма Демукрона, записывая числа в кольцевую очередь на базе связного списка.

Переменные

Константы (#define)

Название переменной	Расшифровка	Тип	Диапазон	Емкость, байт
INITIAL_BUF_SIZE	Начальный размер массива для чтения строк	int	100	4

Структуры данных

Название переменной	Расшифровка	Тип	Диапазон	Емкость (байт)
Node	Узел кольцевой очереди	Структура	-	16
value	Значение в узле	int	от 0 до 2 147 483 647	4
next	Указатель на следующий узел	struct Node*	Адрес памяти	8
CircularQueue	Кольцевая очередь	Структура	-	24
first	Указатель на начало очереди	Node*	Адрес памяти	8
last	Указатель на конец очереди	Node*	Адрес памяти	8
size	Количество элементов в очереди	int	от 0 до 2 147 483 647	4

Функция create_queue()

Название переменной	Расшифровка	Тип	Диапазон	Емкость (байт)
---------------------	-------------	-----	----------	----------------

q	Указатель на новую очередь	CircularQueue*	Адрес памяти	8
---	----------------------------	----------------	--------------	---

Функция enqueue()

Название переменной	Расшифровка	Тип	Диапазон	Емкость (байт)
q	Указатель на очередь	CircularQueue*	Адрес памяти	8
value	Добавляемое значение	int	Зависит от int	4
newNode	Новый узел	Node*	Адрес памяти	8

Функция dequeue()

Название переменной	Расшифровка	Тип	Диапазон	Емкость (байт)
q	Указатель на очередь	CircularQueue*	Адрес памяти	8
temp	Временный узел	Node*	Адрес памяти	8
value	Извлекаемое значение	int	от 0 до 2 147 483 647	4

Функция get_element()

Название переменной	Расшифровка	Тип	Диапазон	Емкость (байт)
q	Указатель на очередь	CircularQueue*	Адрес памяти	8
row	Индекс строки	int	от 0 до 2 147 483 647	4
col	Индекс столбца	int	от 0 до 2 147 483 647	4
n	Размер матрицы	int	от 0 до 2 147 483 647	4
current	Текущий узел	Node*	Адрес памяти	8
pos	Линейная позиция	int	от 0 до 2 147 483 647	4

Функция read_dynamic_line()

Название переменной	Расшифровка	Тип	Диапазон	Емкость (байт)
file	Указатель на файл	FILE*	Адрес памяти	8
buf_size	Размер буфера	size_t	от 0 до 2 ⁶⁴	8
buffer	Буфер для строки	char*	Динамическая память	buf_size
pos	Текущая позиция	size_t	от 0 до 2 ⁶⁴	8
c	Считанный символ	int	от 0 до 2 147 483 647	4
new_buf	Новый буфер	char*	Адрес памяти	8

Функция is_valid_number()

Название переменной	Расшифровка	Тип	Диапазон	Емкость (байт)
str	Проверяемая строка	const char*	Адрес памяти	8
endptr	Указатель на конец числа	char*	Адрес памяти	8

Топологическая сортировка demukron_topological_sort()

Название переменной	Расшифровка	Тип	Диапазон	Емкость (байт)
matrix	Матрица смежности	CircularQueue*	Адрес памяти	8
n	Размер матрицы	int	от 0 до 2 147 483 647	4
result_size	Размер результата	int*	Адрес памяти	8
in_degree	Массив полустепеней захода	int*	от 0 до 2 147 483 647 (1 int)	n * 4
q	Очередь вершин степени 0	CircularQueue*	Адрес памяти	8

result	Результат сортировки	int*	от 0 до 2 147 483 647 (1 int)	n * 4
v	Текущая вершина	int	от 0 до 2 147 483 647	4
i, j	Итераторы	int	от 0 до 2 147 483 647	4
next_level	Очередь следующего уровня	CircularQueue*	Адрес памяти	8

Основная функция main()

Название переменной	Расшифровка	Тип	Диапазон	Емкость (байт)
filename	Имя файла	const char*	“matrix.txt”	Длина строки (10)
file	Указатель на файл	FILE*	Адрес памяти	8
rows	Количество строк	int	от 0 до 2 147 483 647	4
cols	Количество столбцов	int	от 0 до 2 147 483 647	4
line	Считанная строка	char*	Динамическая память	Длина строки
current_cols	Текущее количество столбцов	int	от 0 до 2 147 483 647	4
ptr, token	Указатели для разбора строки	char*	Адрес памяти	8
value	Значение элемента	double	0.0 или 1.0	8
matrix	Матрица смежности	CircularQueue*	Адрес памяти	8
sorted_vertices	Отсортированные вершины	int*	от 0 до 2 147 483 647 (1 int)	n * 4
result_size	Размер результата	int	от 0 до 2 147 483 647	4

Функциональные маршруты

Перед тем как приступить к самой сортировке, необходимо считать из файла матрицу смежности или весов графа. Для топологической сортировки достаточно только матрицы смежности (матрицы, у которой на пересечении i -ой строки и j -ого столбца стоит 1, если существует дуга из вершины i в вершину j , и 0 в противном случае), но также можно рассмотреть матрицу весов, заменяя вес дуги из вершины i в вершину j на 1. Веса никак не влияют на сортировку, поэтому можем от них избавиться и перейти к матрице смежности.

Итак, для того, чтобы перейти к сортировке, нужно проверить следующие свойства матрицы:

1. Матрица состоит из численных значений, не содержит букв и слов;
2. Матрица не пустая;
3. Матрица квадратная, то есть количество столбцов в ней равно количеству строк.

Если в строке присутствуют пустые строки и/или лишние пробелы, то в процессе считывания матрицы из файла в массив необходимо исключить лишние символы.

На первом этапе нужно проверить содержимое файла, чтобы определить, подходит ли оно нам: содержится ли там матрица и удовлетворяет ли она всем вышеописанным условиям.

Если содержимое файла нам подходит, то мы можем записывать значения матрицы в кольцевую очередь, предварительно заменив веса на 1. Матрица будет храниться в виде длинного односвязного списка, поэтому для дальнейшей работы с ней нужно запомнить ее размерность.

Когда матрица была успешно считана и записана в очередь, можно переходить к самой топологической сортировке с помощью алгоритма Демукрона.

Топологическая сортировка — это процесс упорядочивания вершин ациклического ориентированного графа согласно частичному порядку, заданному рёбрами орграфа на множестве его вершин. Метод Демукрона — это алгоритм топологической сортировки, который использует разбиение графа на уровни.

Алгоритм топологической сортировки Демукрона:

1. Вычислить полустепеней захода: для каждой вершины $v \in V$ вычисляем количество входящих ребер.
2. Найти все вершины с нулевой полустепенью захода.

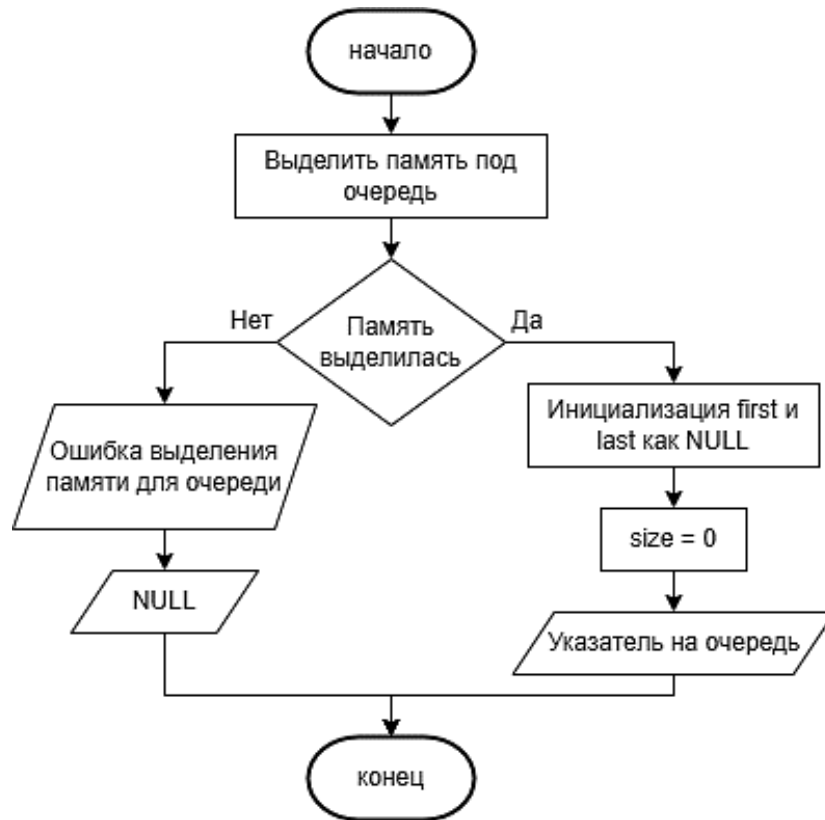
3. Добавить их в текущий уровень и удалить из графа.
4. Обновить полустепени захода для всех вершин, в которые входят рёбра из удалённых вершин.
5. Повторять пункты 2 – 4, пока все вершины не будут распределены по уровням.

Если после обработки остались вершины с ненулевыми полустепенями захода, значит, граф содержит цикл, и топологическая сортировка невозможна.

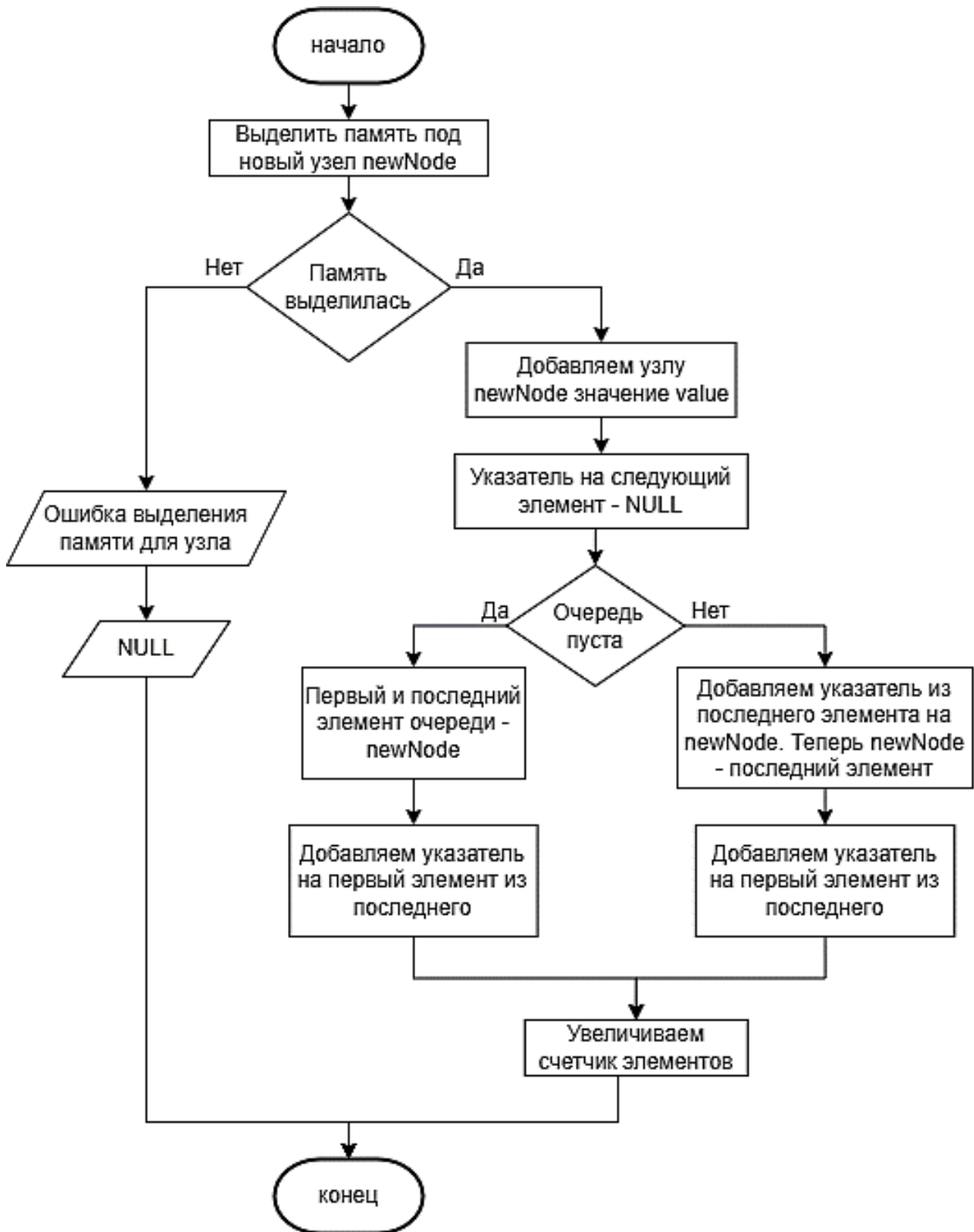
После последовательного объединения уровней (первый + второй + ...) получится отсортированный список вершин. Если на одном уровне находится несколько вершин, то в списке их можно располагать в произвольном порядке.

Блок-схемы

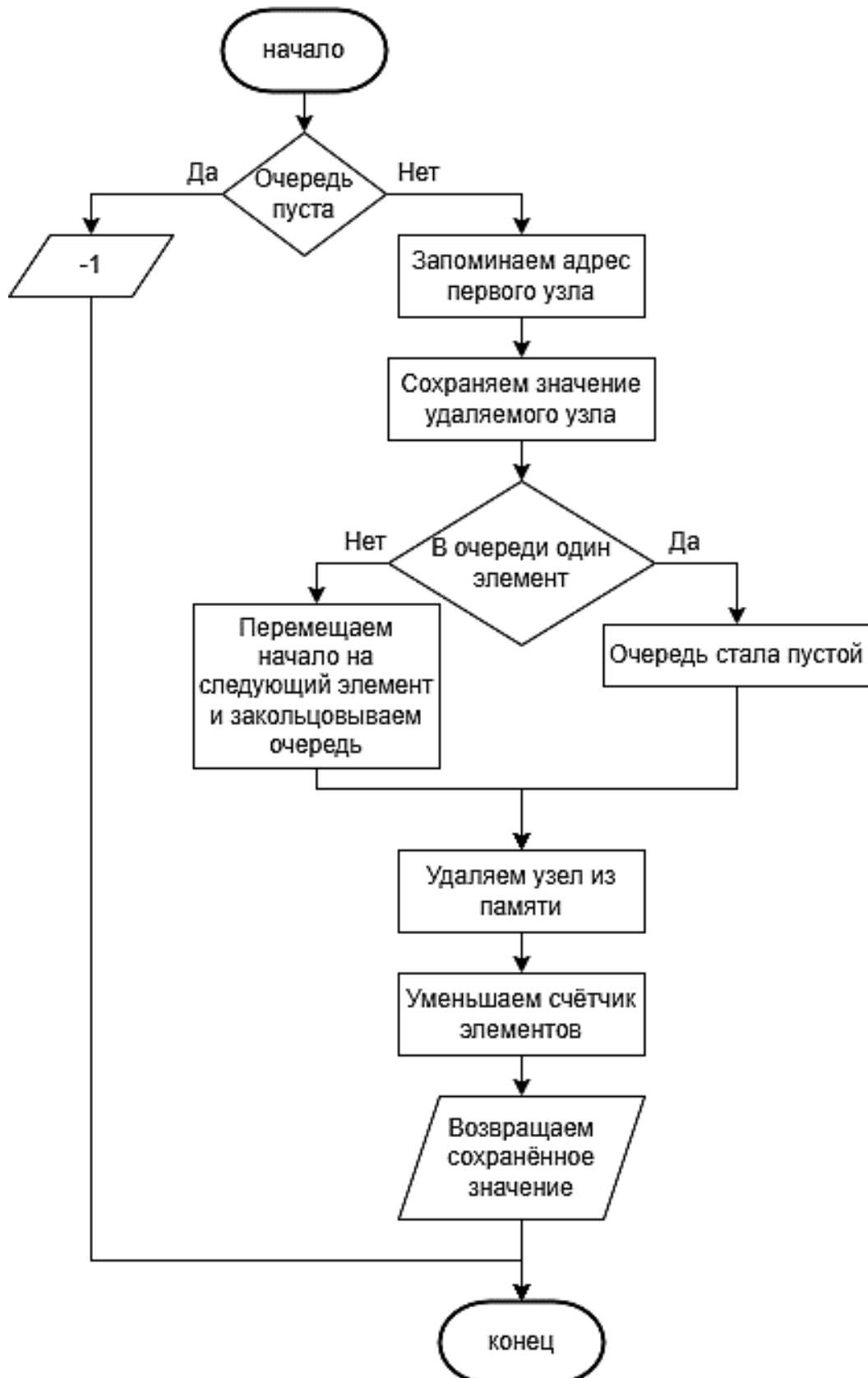
Функция `create_queue()`



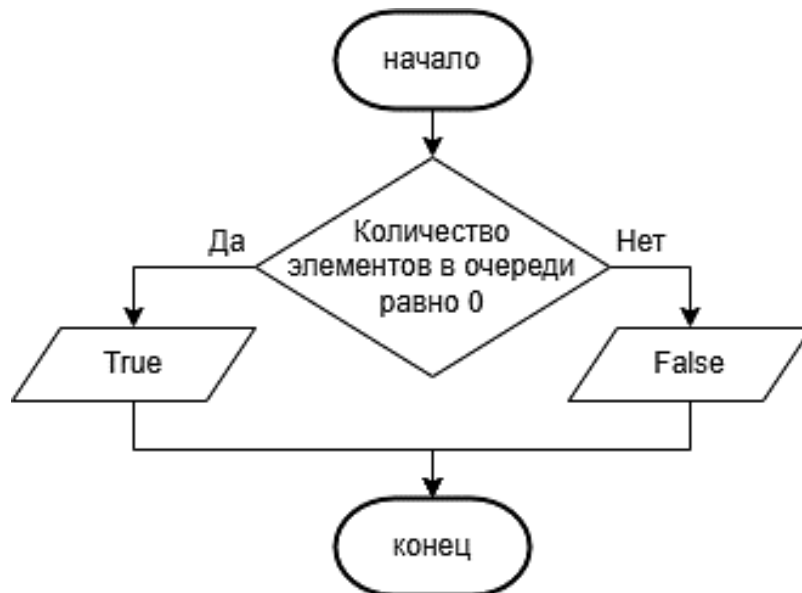
Функция **enqueue(CircularQueue* q, int value)**



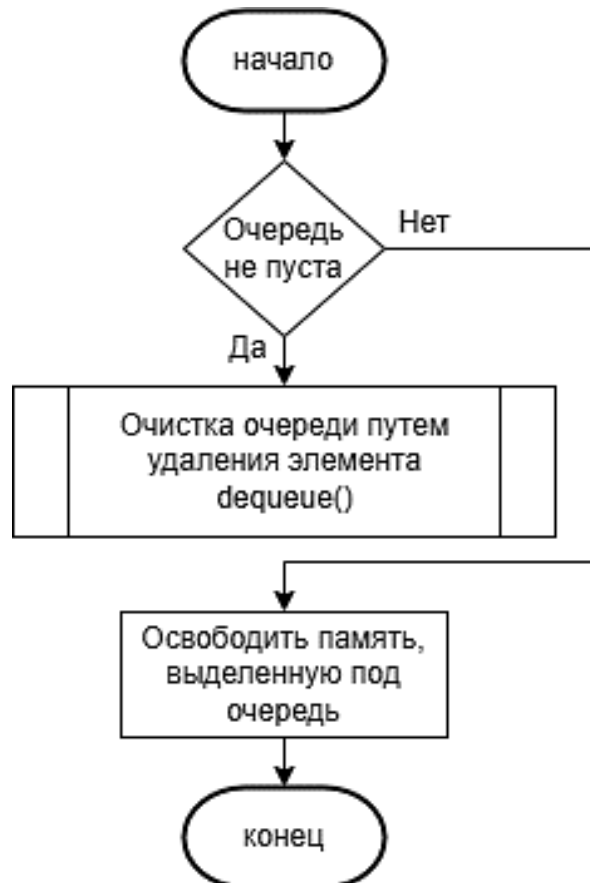
Функция **dequeue(CircularQueue* q)**



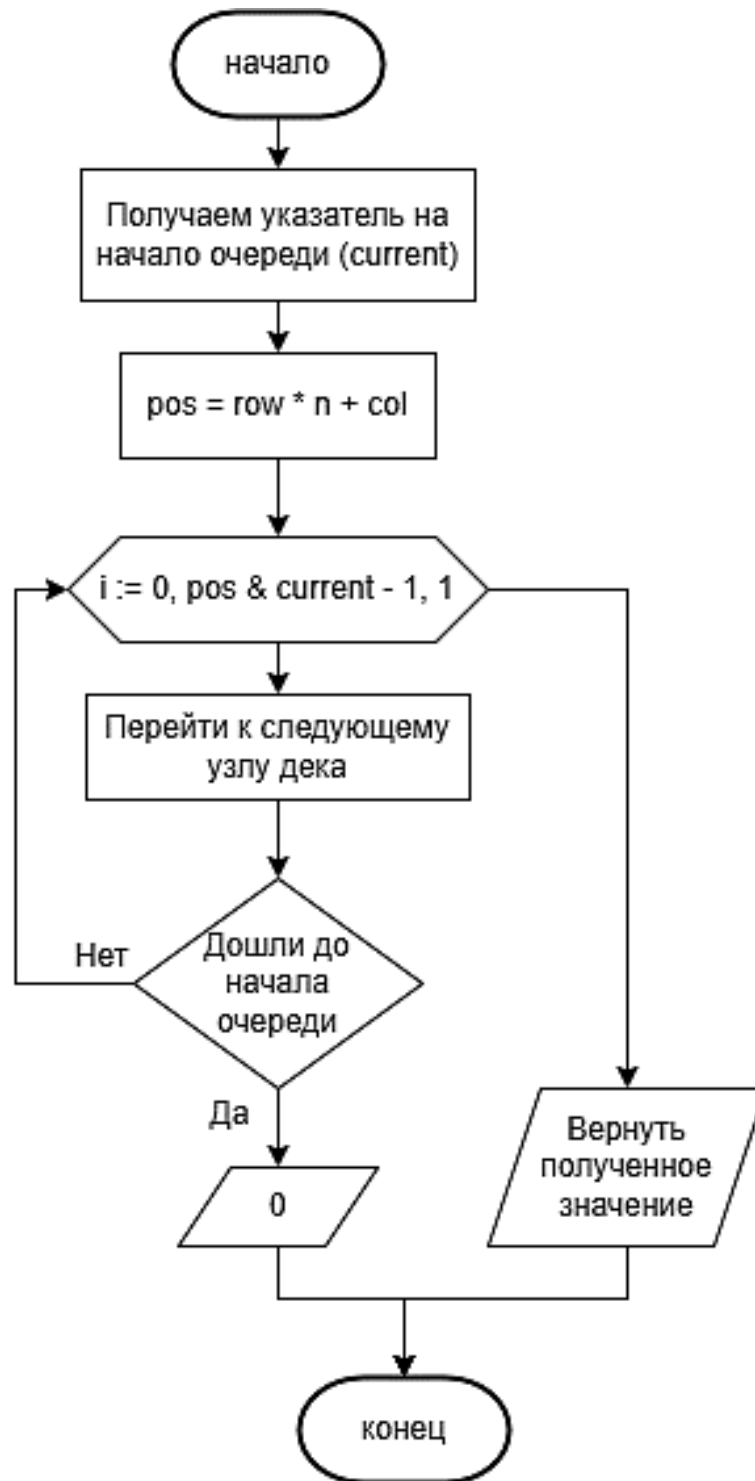
Функция **is_empty(const CircularQueue* q)**



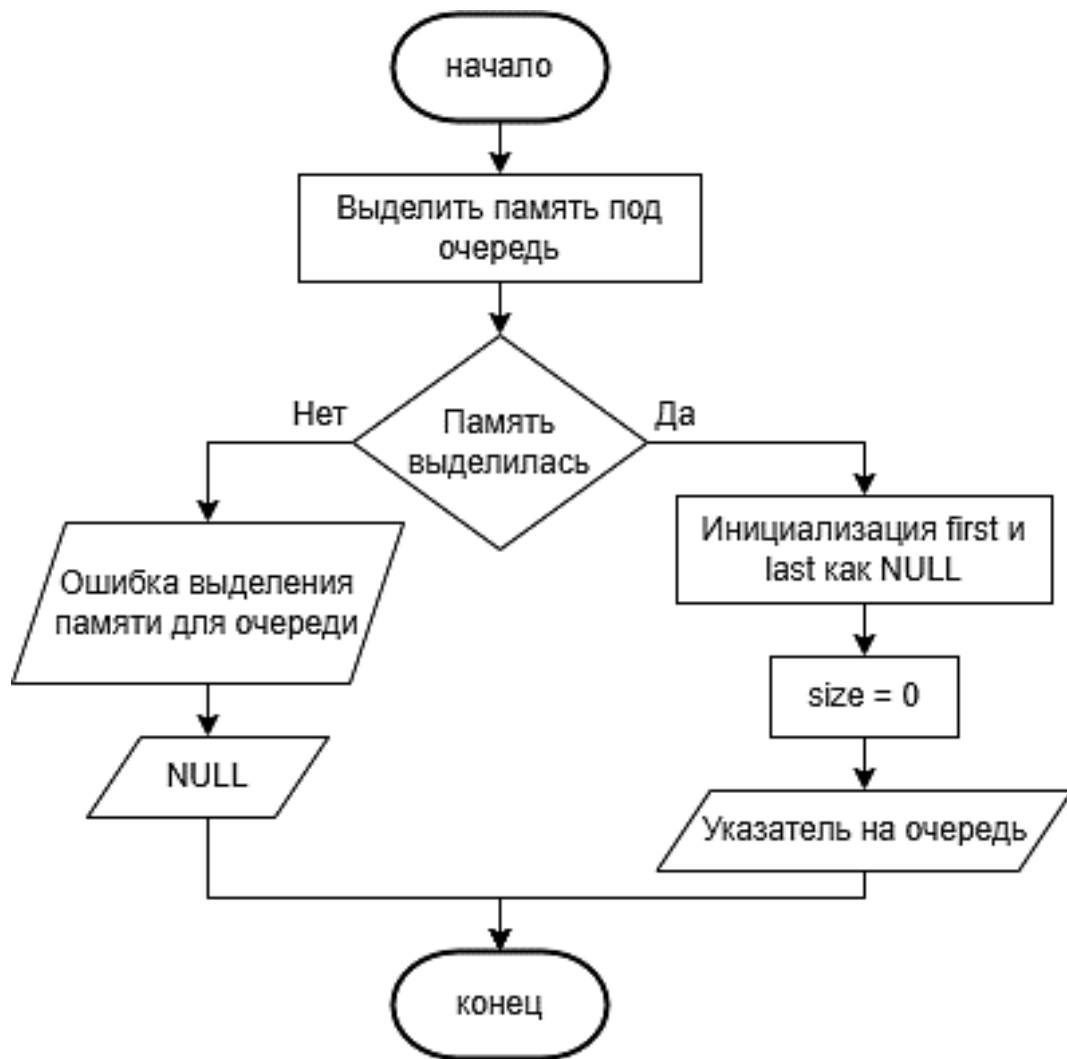
Функция **free_queue(CircularQueue* q)**



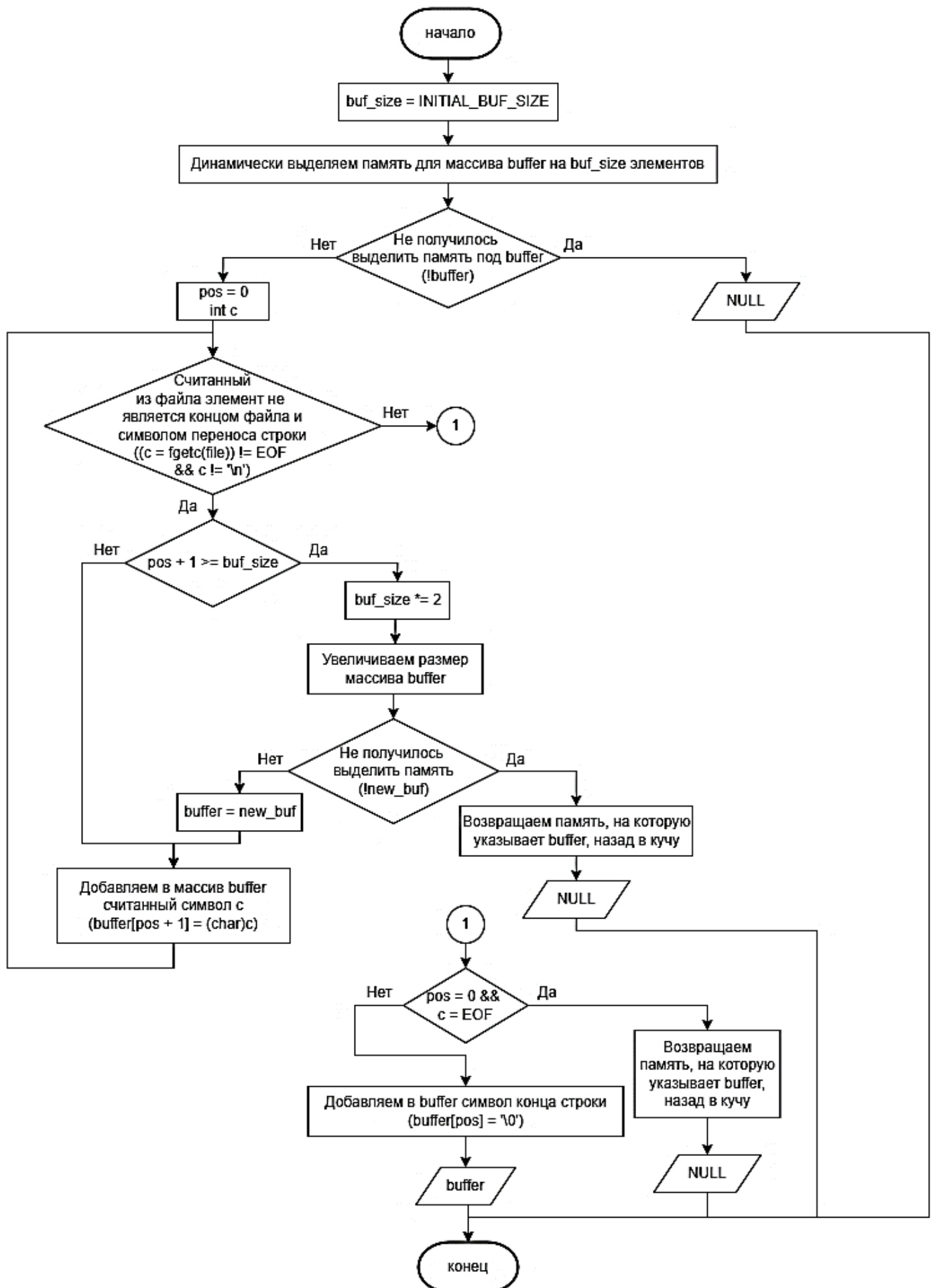
Функция `get_element(const CircularQueue* q, int row, int col, int n)`



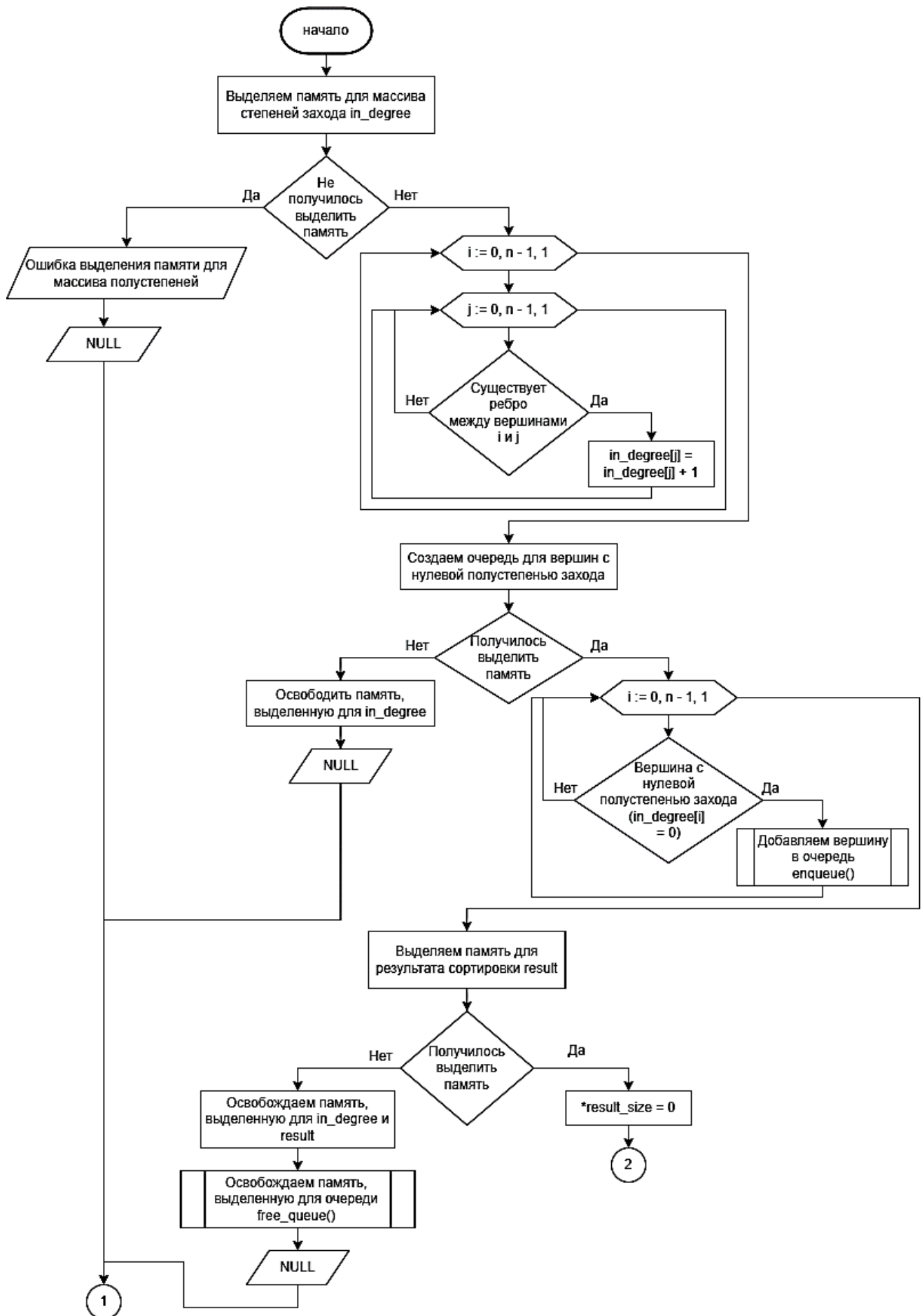
Функция `is_valid_number(const char *str)`

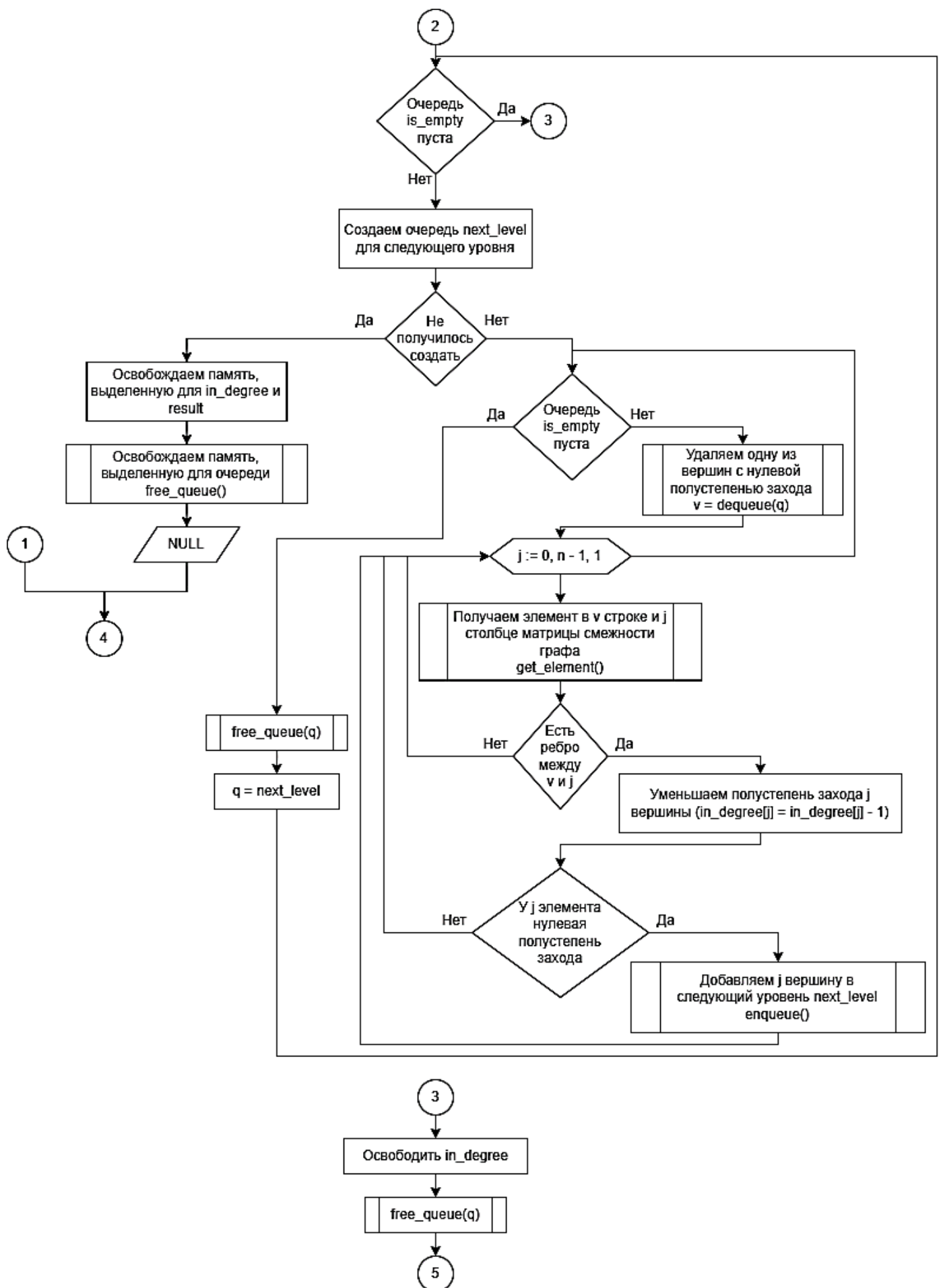


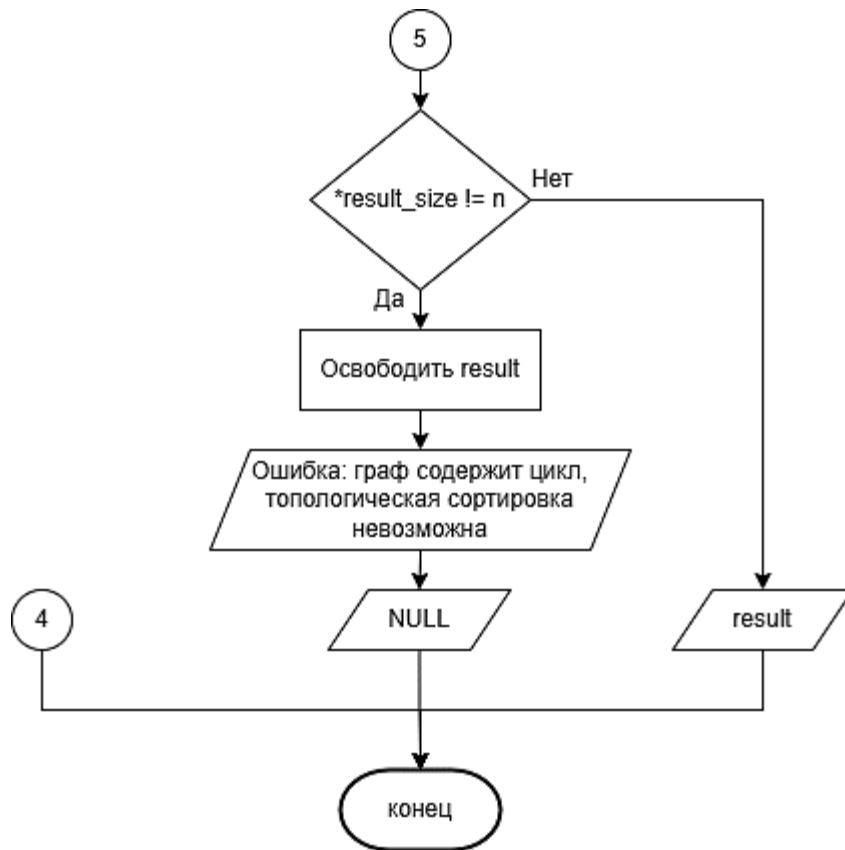
Функция `read_dynamic_line(FILE* file)`



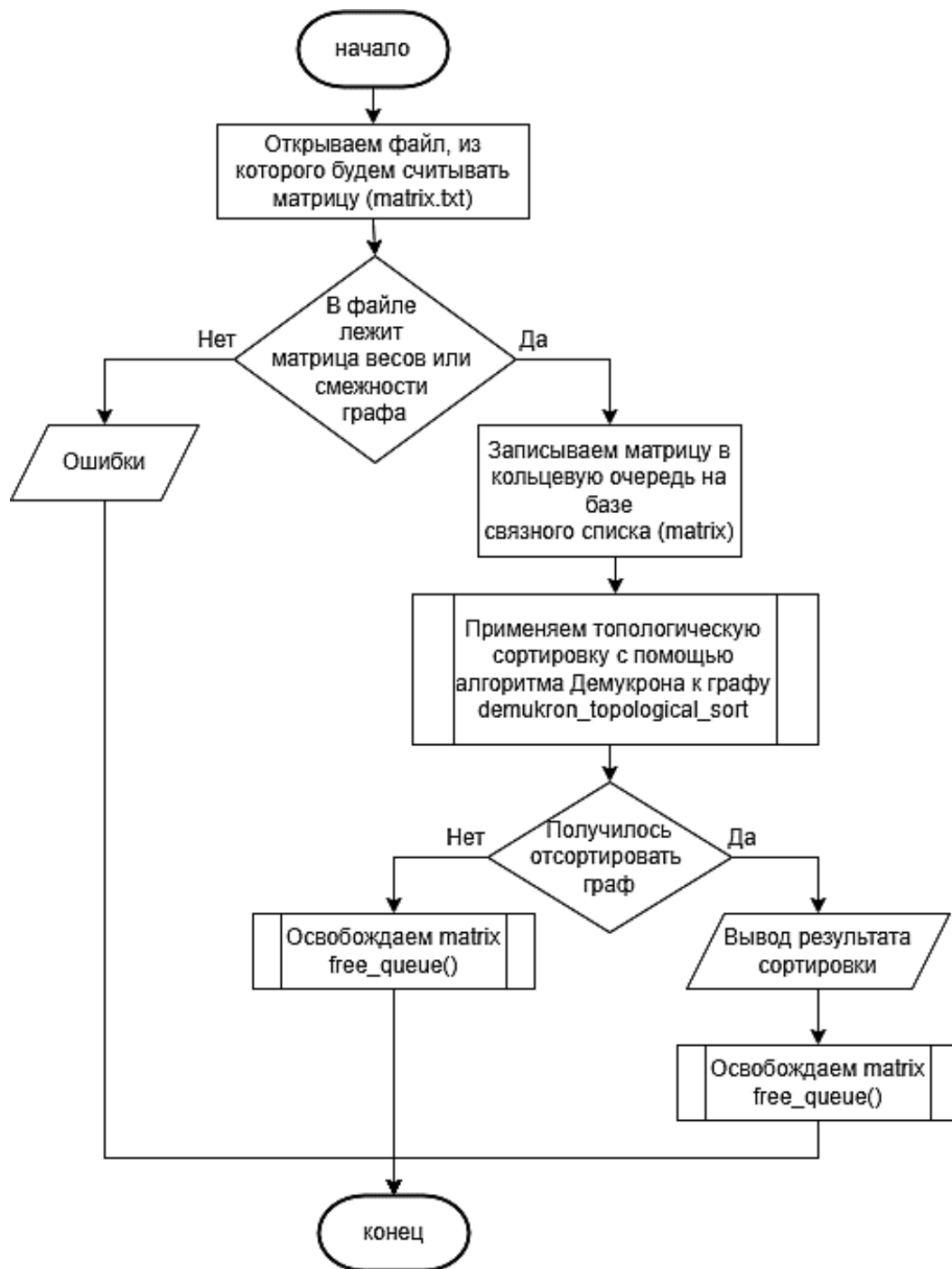
Функция **demukron_topological_sort(const CircularQueue* matrix, int n, int* result_size)**







Функция **main()**



Код на языке программирования C

```
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <errno.h>

#define INITIAL_BUF_SIZE 100 // Начальный размер буфера для чтения строк

// Структура узла кольцевой очереди
typedef struct Node {
    int value; // Значение, которое лежит в узле
    struct Node* next; // Указатель на следующий узел
} Node;

// Структура кольцевой очереди
typedef struct CircularQueue {
    Node* first; // Указатель на начало очереди
    Node* last; // Указатель на конец очереди
    int size; // Текущее количество элементов
} CircularQueue;

// Создает новую пустую кольцевую очередь
CircularQueue* create_queue() {
    CircularQueue* q = malloc(sizeof(CircularQueue));
    if (!q) {
        perror("Ошибка выделения памяти для очереди");
        return NULL;
    }
    q->first = q->last = NULL;
    q->size = 0;
    return q;
}

// Добавляет элемент в конец очереди
void enqueue(CircularQueue* q, int value) {
    Node* newNode = malloc(sizeof(Node));
    if (!newNode) {
        perror("Ошибка выделения памяти для узла");
        return;
    }

    newNode->value = value;
    newNode->next = NULL;

    if (q->last == NULL) {
        // Очередь пуста - новый элемент становится и началом, и концом
    }
}
```

```

    q->first = q->last = newNode;
    q->last->next = q->first; // Замыкаем кольцо
} else {
    // Добавляем элемент в конец очереди
    q->last->next = newNode;
    q->last = newNode;
    newNode->next = q->first; // Замыкаем кольцо
}
q->size++;
}

// Удаляет элемент из начала очереди и возвращает его значение
int dequeue(CircularQueue* q) {
    if (!q || q->first == NULL) return -1; // Очередь пуста

    Node* temp = q->first;
    int value = temp->value;

    if (q->first == q->last) {
        // В очереди только один элемент
        q->first = q->last = NULL;
    } else {
        // Перемещаем начало на следующий элемент
        q->first = q->first->next;
        q->last->next = q->first; // Обновляем ссылку для кольца
    }

    free(temp);
    q->size--;
    return value;
}

// Проверяет, пуста ли очередь
bool is_empty(const CircularQueue* q) {
    return !q || q->size == 0;
}

// Освобождает память, занятую очередью
void free_queue(CircularQueue* q) {
    while (!is_empty(q)) {
        dequeue(q);
    }
    free(q);
}

// Получает элемент матрицы по индексам строки и столбца
int get_element(const CircularQueue* q, int row, int col, int n) {
    Node* current = q->first;

```

```

int pos = row * n + col; // Линейная позиция в матрице

// Переходим к нужной позиции
for (int i = 0; i < pos; i++) {
    current = current->next;
    if (current == q->first) {
        // Защита от бесконечного цикла
        return 0;
    }
}

return current->value;
}

// Проверяет, является ли строка корректным числом
bool is_valid_number(const char *str) {
    if (!str || *str == '\0') return false;

    char *endptr;
    errno = 0;
    strtod(str, &endptr);

    // Проверяем:
    // 1. Преобразование завершилось на конце строки или пробеле
    // 2. Не было переполнения
    return (*endptr == '\0' || isspace(*endptr)) && errno != ERANGE;
}

// Читает строку произвольной длины из файла
char* read_dynamic_line(FILE* file) {
    if (!file) return NULL;

    size_t buf_size = INITIAL_BUF_SIZE;
    char* buffer = malloc(buf_size);
    if (!buffer) {
        perror("Ошибка выделения памяти для буфера");
        return NULL;
    }

    size_t pos = 0;
    int c;

    while ((c = fgetc(file)) != EOF && c != '\n') {
        if (pos + 1 >= buf_size) {
            // Увеличиваем буфер при необходимости
            buf_size *= 2;
            char* new_buf = (char*)realloc(buffer, buf_size);
            if (!new_buf) {

```

```

        free(buffer);
        perror("Ошибка перевыделения памяти для буфера");
        return NULL;
    }
    buffer = new_buf;
}
buffer[pos++] = (char)c;
}

if (pos == 0 && c == EOF) {
    // Не прочитали ни одного символа
    free(buffer);
    return NULL;
}

buffer[pos] = '\0'; // Завершаем строку
return buffer;
}

// Алгоритм топологической сортировки Демукрона
int* demukron_topological_sort(const CircularQueue* matrix, int n, int* result_size) {
    // Выделяем память для массива полустепеней захода
    int* in_degree = calloc(n, sizeof(int));
    if (!in_degree) {
        perror("Ошибка выделения памяти для массива полустепеней");
        return NULL;
    }

    // Вычисляем полустепени захода для всех вершин
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (get_element(matrix, i, j, n)) {
                in_degree[j]++;
            }
        }
    }

    // Создаем очередь для вершин с нулевой полустепенью захода
    CircularQueue* q = create_queue();
    if (!q) {
        free(in_degree);
        return NULL;
    }

    // Добавляем в очередь вершины с нулевой полустепенью захода
    for (int i = 0; i < n; i++) {
        if (in_degree[i] == 0) {
            enqueue(q, i);
        }
    }
}

```

```

    }
}

// Выделяем память для результата сортировки
int* result = malloc(n * sizeof(int));
if (!result) {
    free(in_degree);
    free_queue(q);
    perror("Ошибка выделения памяти для результата");
    return NULL;
}

*result_size = 0; // Изначально результат пуст

// Основной цикл алгоритма
while (!is_empty(q)) {
    // Создаем очередь для следующего уровня
    CircularQueue* next_level = create_queue();
    if (!next_level) {
        free(in_degree);
        free_queue(q);
        free(result);
        return NULL;
    }

    // Обработываем все вершины текущего уровня
    while (!is_empty(q)) {
        int v = dequeue(q);
        result[(*result_size)++] = v; // Добавляем вершину в результат

        // Уменьшаем полустепени захода для всех соседей
        for (int j = 0; j < n; j++) {
            if (get_element(matrix, v, j, n)) {
                in_degree[j]--;
                if (in_degree[j] == 0) {
                    enqueue(next_level, j);
                }
            }
        }
    }

    // Переходим к следующему уровню
    free_queue(q);
    q = next_level;
}

// Освобождаем ресурсы
free(in_degree);

```

```

free_queue(q);

// Проверяем наличие циклов
if (*result_size != n) {
    free(result);
    printf("Ошибка: граф содержит цикл, топологическая сортировка невозможна\n");
    return NULL;
}

return result;
}

int main() {
    const char* filename = "matrix.txt";
    FILE* file = fopen(filename, "r");
    if (!file) {
        perror("Ошибка открытия файла");
        return 1;
    }

    int rows = 0, cols = 0;
    char* line = NULL;

    // Первый проход: определяем размеры матрицы
    while ((line = read_dynamic_line(file)) != NULL) {
        if (strlen(line) == 0) {
            free(line);
            continue;
        }

        int current_cols = 0;
        char* ptr = line;
        char* token;

        // Разбиваем строку на токены
        while (*ptr) {
            // Пропускаем пробелы
            while (*ptr && isspace(*ptr)) ptr++;
            if (!*ptr) break;

            token = ptr;
            // Переходим к концу числа
            while (*ptr && !isspace(*ptr)) ptr++;

            // Проверяем, является ли токен числом
            if (!is_valid_number(token)) {
                printf("Ошибка: некорректное значение '%s'\n", (int)(ptr - token), token);
                free(line);
            }
        }
    }
}

```

```

        fclose(file);
        return 1;
    }
    current_cols++;
}

if (cols == 0) {
    cols = current_cols; // Первая строка задает количество столбцов
} else if (current_cols != cols) {
    printf("Ошибка: разное количество элементов в строках\n");
    free(line);
    fclose(file);
    return 1;
}
rows++;
free(line);
}

// Проверка на пустую матрицу
if (rows == 0 || cols == 0) {
    printf("Ошибка: пустая матрица\n");
    fclose(file);
    return 1;
}

// Проверка на квадратность матрицы
if (rows != cols) {
    printf("Ошибка: матрица смежности должна быть квадратной\n");
    fclose(file);
    return 1;
}

// Возвращаемся в начало файла для второго прохода
rewind(file);

// Создаем матрицу смежности в виде кольцевой очереди
CircularQueue* matrix = create_queue();
if (!matrix) {
    fclose(file);
    return 1;
}

// Второй проход: заполняем матрицу
while ((line = read_dynamic_line(file)) != NULL) {
    if (strlen(line) == 0) {
        free(line);
        continue;
    }
}

```

```

char* ptr = line;
char* token;

while (*ptr) {
    while (*ptr && isspace(*ptr)) ptr++;
    if (!*ptr) break;

    token = ptr;
    while (*ptr && !isspace(*ptr)) ptr++;

    // Преобразуем строку в число и записываем в матрицу
    double value = strtod(token, NULL);
    if (value == 0) enqueue(matrix, 0);
    else enqueue(matrix, 1);
}
free(line);
}
fclose(file);

// Выводим матрицу смежности для проверки
printf("Матрица смежности графа (%dx%d):\n", rows, cols);
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        printf("%d ", get_element(matrix, i, j, rows));
    }
    printf("\n");
}

// Выполняем топологическую сортировку
int result_size;
int* sorted_vertices = demukron_topological_sort(matrix, rows, &result_size);

if (!sorted_vertices) {
    free_queue(matrix);
    return 1;
}

// Выводим результат сортировки
printf("Топологически отсортированные вершины: ");
for (int i = 0; i < result_size; i++) {
    printf("%d ", sorted_vertices[i] + 1); // +1 для нумерации с 1
}
printf("\n");

// Освобождаем память
free(sorted_vertices);
free_queue(matrix);

```

```
    return 0;  
}
```

Примеры работы программы

C: > Users > Анастасия > ≡ matrix.txt

```
1  0 0 0
2  1 0 0
3  1 1 0
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Анастасия> gcc lab_3_algorithms.c -o program

PS C:\Users\Анастасия> ./program

Матрица смежности графа (3x3):

0 0 0

1 0 0

1 1 0

Топологически отсортированные вершины: 3 2 1

C: > Users > Анастасия > ≡ matrix.txt

```
1
2  |   0   102   1  -0.123   0.1
3
4  0 0   0 -5 0
5  0           0 0 12568979 1
6  0 0 0 0 1.025956565
7  0  0   0 0 0
8
9
10
11
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Анастасия> gcc lab_3_algorithms.c -o program

PS C:\Users\Анастасия> ./program

Матрица смежности графа (5x5):

0 1 1 1 1

0 0 0 1 0

0 0 0 1 1

0 0 0 0 1

0 0 0 0 0

Топологически отсортированные вершины: 1 2 3 4 5